

TP électronique M1
2025

Microcontrôleur STM32/Cortex M3 et langage assembleur

Jean-Jacques MENEU. Alain REBOUX

Chapitre 1. Introduction.....	3
Chapitre 2. Découverte du microcontrôleur et de l'IDE	4
2.1. Présentation du microcontrôleur	4
2.2. Programmation en langage assembleur.....	8
2.3. Instructions du STM32	14
2.4. Modes d'adressage du STM32	14
2.5. Ensemble de développement.....	16
Chapitre 3. Premiers pas	17
3.1. Introduction	17
3.2. Réalisation d'un compteur modulo 10.....	27
3.3. Comment faire une boucle "for" ?.....	28
3.4. Mixer assembleur et C, Clignotement d'une LED avec ODR et BSRR.....	29
Annexe 1. Les instructions saut conditionnel.....	32
Annexe 2. Créer un projet STM32	33
Annexe 3. Fenêtre de débogage	37

Chapitre 1. Introduction

L'objectif de ce TP qui intervient après la série de TP A3 en langage C est une brève initiation au langage assembleur. Vous découvrirez l'utilisation des registres internes du microprocesseur CORTEX M3, quelques instructions essentielles et différents modes d'adressage.

L'environnement de développement sera comme l'année dernière STM32CubeIDE qui intègre tous les outils de développement d'un logiciel embarqué (gestionnaire de projet, éditeur avec coloriage syntaxique, assembleur, éditeur de lien, débogueur symbolique, carte de développement du STM32 appelé Nucleo-L152),

Le temps est limité, il n'y a que 3 heures prévues pour cette initiation. Il est essentiel de préparer ce TP. La séance de TP commencera par la présentation par l'enseignant de la correction de l'exercice 3.2 "Réalisation d'un compteur modulo 10"

Préparation à faire avant séance de TP

- Lecture du chapitre 2
- Réalisation de l'exercice 3.1
- Téléchargement des fichiers suivants disponible sur l'ENT:
 - startup_stm32l152retx.s
 - fonc_asm.h
 - fonc_asm.s

Chapitre 2. Découverte du microcontrôleur et de l'IDE

2.1. Présentation du microcontrôleur

Un microcontrôleur est un microprocesseur ou CPU autour duquel ont été rajoutés des mémoires et des périphériques.

La figure ci-dessous présente le microcontrôleur STM32L152RE.

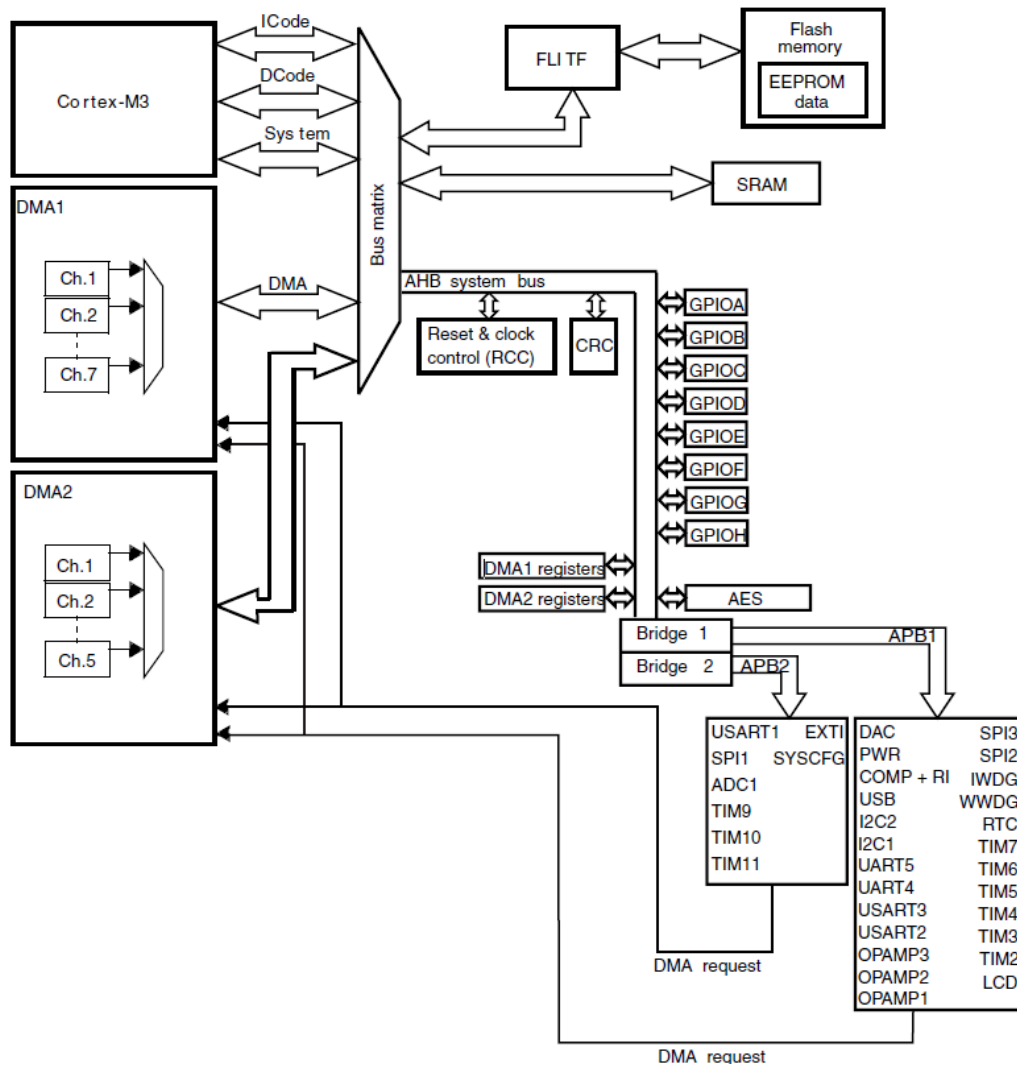


Figure 1. Schéma block du STM32L152

L'architecture du **CPU** est de type HARVARD avec un bus d'adresse de 32 bits vers la mémoire programme et un bus d'adresse de 32 bits entre la mémoire et les données..

Le **CPU** contient 18 registres tous de taille 32 bits :

- douze registres généraux (R0 à R12)
- un compteur ordinal PC (Program Counter),
- Un pointeur de pile SP (Stack Pointer)
- Une registre de lien LR (Link Register)
- un registre d'état PSR (Program Status Register),

- un pointeur de pile SP (16 bits).

La **mémoire** présente est de type :

- Flash (256 Ko) pour la mémoire programme,
- RAM (80Ko),
- EEPROM (16Ko).

Les **périphériques** sont :

- ports d'entrées/sorties (PORT A, PORT B, PORT C, PORT D) avec un nombre de broches qui varient
- Watchdog : IWDG (Independant Watchdog) et WWDG (Window Watchdog)
- Timers : timers basiques et avancés, 32 ou 16-bit
- module de communication I2C
- module de communication SPI
- module de communication UART
- convertisseur analogique numérique
- convertisseur numérique analogique
- des fonctions analogiques comme des comparateurs et des amplificateurs opérationnels
- ...

L'accès aux périphériques se fait à travers différents bus : AHB, APB1 et APB2. Vous ferez attention de gérer l'horloge pour chaque périphérique.

L'utilisation des registres des périphériques est expliquée dans la documentation technique du µcontrôleur (reference Manual) et bien sûr pendant les cours.

Dans les systèmes à µcontrôleur, différents types de mémoires sont utilisés selon leur fonction : la mémoire non volatile (ROM ou Flash) pour le programme et les constantes, la mémoire volatile à lecture et écriture (RAM) pour les données temporaires. Cependant vue du programmeur, la mémoire forme un unique espace adressable continu, comme le montre la figure suivante :

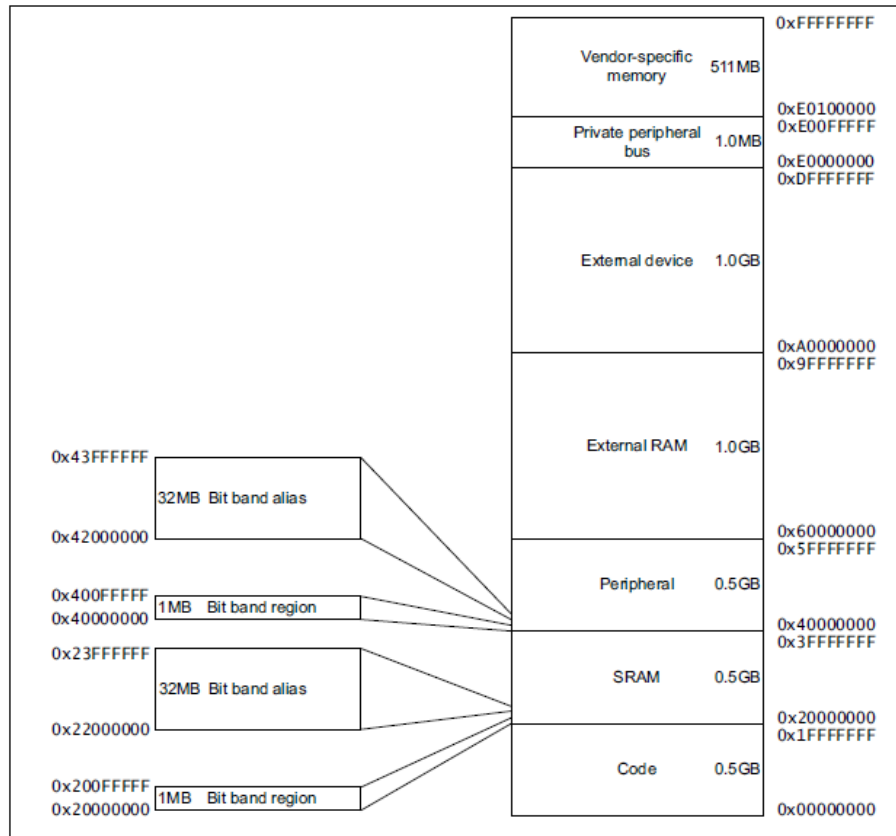


Figure 1 : organisation de l'espace adressable

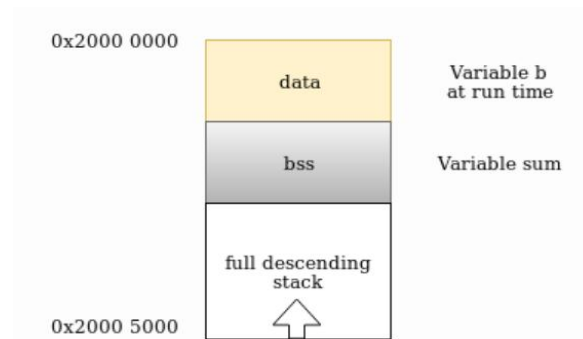


Figure 3 : organisation de l'espace RAM

On peut voir dans cette structure trois espaces **RAM** différents La RAM est composée de trois espaces différents:

1. le premier espace correspond aux variables qui sont initialisées dans le code. Cet espace, appelé data, commence toujours à 0x2000 0000 et est ascendant
2. le second espace bss correspond à des variables qui ne sont pas initialisés mais dont on a réservé un espace mémoire. Son adresse de départ dépend de la taille de la section data.
3. le troisième espace correspond à la **pile** (Stack). L'adresse de départ de la pile peut être programmée et est descendante. La valeur de départ maximum dépend de la taille SRAM. Avec 80Ko, cette valeur est égale à 0x2001 4000

Les périphériques sont désignés par un nom et correspondent en réalité à l'accès à des registres particuliers se trouvant dans les adresses mémoire à partir de 0x4000 0000.

Les vecteurs d'interruption démarrent à l'adresse 0x0000 0004 de la mémoire programme. Ils définissent les adresses des sous-programmes à exécuter lorsqu'une interruption arrive. Par exemple l'interruption due au Reset du µcontrôleur est liée à l'adresse du début du programme principal (main). Ainsi, dès que le µcontrôleur est mis sous tension, l'interruption du reset est générée, et cela force le CPU à sauter à l'adresse du début du programme pour l'exécuter.

Un tableau donne la correspondance entre les vecteurs d'interruptions (le nombre d'interruptions dépend des microcontrôleurs. ARM permet un maximum de 256 interruptions, mais la vaste majorité des microcontrôleurs possèdent beaucoup de vecteurs d'interruption. Le STM32L152RE possède 56 interruptions..

Exception number	IRQ number	Offset	Vector
83	67	0x014C	IRQ67
.	.	.	.
.	.	.	.
.	.	.	.
18	2	0x004C	IRQ2
17	1	0x0048	IRQ1
16	0	0x0044	IRQ0
15	-1	0x0040	Systick
14	-2	0x003C	PendSV
13		0x0038	Reserved
12			Reserved for Debug
11	-5	0x002C	SVCall
10			Reserved
9			
8			
7			
6	-10	0x0018	Usage fault
5	-11	0x0014	Bus fault
4	-12	0x0010	Memory management fault
3	-13	0x000C	Hard fault
2	-14	0x0008	NMI
1		0x0004	Reset
		0x0000	Initial SP value

Figure 4 : Table des vecteurs d'interruption

Table 51. Vector table (Cat.4, Cat.5 and Cat.6 devices)

Position	Priority	Type of priority	Acronym	Description	Address
-	-	-	-	Reserved	0x0000_0000
-	-3	fixed	Reset	Reset	0x0000_0004
-	-2	fixed	NMI_Handler	Non maskable interrupt. The RCC Clock Security System (CSS) is linked to the NMI vector.	0x0000_0008
-	-1	fixed	HardFault_Handler	All class of fault	0x0000_000C
-	0	settable	MemManage_Handler	Memory management	0x0000_0010
-	1	settable	BusFault_Handler	Pre-fetch fault, memory access fault	0x0000_0014
-	2	settable	UsageFault_Handler	Undefined instruction or illegal state	0x0000_0018
-	-	-	-	Reserved	0x0000_001C - 0x0000_002B
-	3	settable	SVC_Handler	System service call via SWI instruction	0x0000_002C
-	4	settable	DebugMon_Handler	Debug Monitor	0x0000_0030
-	-	-	-	Reserved	0x0000_0034
-	5	settable	PendSV_Handler	Pendable request for system service	0x0000_0038
-	6	settable	SysTick_Handler	System tick timer	0x0000_003C
0	7	settable	WWDG	Window Watchdog interrupt	0x0000_0040
1	8	settable	PVD	PVD through EXTI Line16 detection interrupt	0x0000_0044
2	9	settable	TAMPER_STAMP	Tamper, LSECSS and TimeStamp through EXTI line19 interrupts	0x0000_0048
3	10	settable	RTC_WKUP	RTC Wakeup through EXTI line20 interrupt	0x0000_004C
4	11	settable	FLASH	Flash global interrupt	0x0000_0050
5	12	settable	RCC	RCC global interrupt	0x0000_0054
6	13	settable	EXTI0	EXTI Line0 interrupt	0x0000_0058
7	14	settable	EXTI1	EXTI Line1 interrupt	0x0000_005C
8	15	settable	EXTI2	EXTI Line2 interrupt	0x0000_0060
9	16	settable	EXTI3	EXTI Line3 interrupt	0x0000_0064

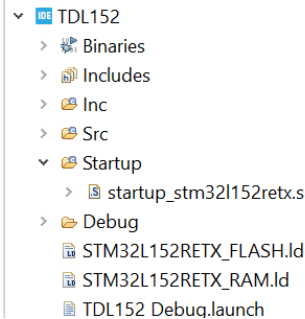
Figure 4 : Table des vecteurs d'interruption avec adresse

2.2. Programmation en langage assembleur

2.2.1. Squelette de programme

La programmation du μ contrôleur se fait en langage assembleur suivant les instruction Thumb-2, ARMV7. L'assembleur comporte un jeu d'instructions complet et de nombreux modes d'adressage.

Le programme de démarrage est toujours écrit en langage assembleur. Lorsque STM32CubeIDE initialise un projet, un fichier startup_stm32l152retx.s est créé. Un fichier .s signifie que le fichier contient du code assembleur.



Les directives dans ce fichier suivent les normes gcc. Il n'est pas nécessaire de connaître ces directives pour le TP. Un autre fichier vous est fourni avec des indications où écrire le code et déclarer les variables afin de vous concentrer uniquement sur le code assembleur. Après la création d'un projet, remplacer le fichier de startup par celui fourni.

Le squelette du programme " stm32l152retx.s " fourni a pour but de bien définir la structure d'un programme et les différentes zones mémoires nécessaires pour la construction du code exécutable. Il servira de base pour l'écriture de vos programmes et comporte plusieurs zones.

2.2.1.1. Zone de description

Cette zone est constituée de commentaires et permet de décrire ce que fait le programme. Il est très important de commenter les programmes, quel que soit le langage de programmation, mais c'est particulièrement vrai en langage assembleur, car c'est un langage peu lisible.

La syntaxe des commentaires est la suivante :

```
/* ceci est un commentaire sur une ligne */
          INST                // commentaire d'une instruction
```

2.2.1.2. Zone de déclaration des symboles

Cette zone permet de déclarer des symboles pour simplifier la lecture du code source. Ils sont désignés par un nom (on dit également label) et ne prennent pas de place en mémoire (contrairement aux constantes). Lors de la compilation, le label est automatiquement remplacé par la valeur qu'il désigne. C'est l'équivalent d'un "#define" en C.

La syntaxe est la suivante :

```
.equ  nom, valeur
```

Par exemple :

```
.~
47 /*****
48 *
49 *          AREA TO DECLARE SYMBOLS
50 *
51 /*****/
52 .equ myvar, 0x87654321
53
54 /*****
55 *
56 *          END  AREA TO DECLARE STMBOLS
57 *
58 /*****/
```

Remarques :

- Pensez à donner des noms clairs afin de simplifier la lecture du code. Cette remarque est valable pour tous les types de noms que vous définirez : symboles, variables, constantes, sous-programmes.
- Pour la lisibilité de votre code, utilisez la convention suivante où [TAB] désigne la touche de tabulation :

Label :

[TAB]INST [TAB]opérande[TAB] // commentaire

2.2.1.3. Zone de déclaration des variables

Cette zone se trouve dans la RAM de la mémoire.

Chaque variable est déclarée par un nom (ou label) et une taille en nombre d'octets ou de mots(word) (1 mot = 4 octets). Pour simplifier le programme, nous ne déclarerons que des mots. On peut ainsi déclarer également des tableaux et des chaînes de caractère.

La syntaxe est la suivante :

Nom de variable :

.word valeur

Et un pour un tableau

Nom de variable :

.word valeur1, valeur2, valeur3

Quelques exemples de déclaration :

```
/******  
*  
*          AREA TO DECLARE INITIALIZED VARIABLES          *  
*  
/******/  
.section .data  
  
CPT:  
    .word 10  
  
varindata:  
    .word 0x12345678, 0x0A, 0xDEADBEEF  
    .string "hello word"  
  
decalage:  
    .word 0x3  
  
/******  
*  
*          END    AREA TO DECLARE INITIALIZED VARIABLES    *  
*  
/******/
```

De manière similaire, il est également possible de déclarer des variables non initialisées dans la section bss.

2.2.1.4. Zone de déclaration des constantes

Cette zone se trouve dans la zone mémoire programme (Flash). Chaque constante est déclarée par un nom (ou label) et une valeur. On peut aussi déclarer des tableaux de constantes.

```

/*****
*
*          AREA TO DECLARE CONSTANTS
*
*****/
.section .text
varconstant:
    .word 0xDEADBEEF

/*****
*
*          END    AREA TO DECLARE CONSTANTS
*
*****/

```

Afin de simplifier le programme, déclarer seulement des word qui correspondent à des variables de longueur 32 bits.

La syntaxe est la suivante :

Nom de variable :

.word valeur

Et un pour un tableau

Nom de variable :

.word valeur1, valeur2, valeur3

2.2.1.5. Zone de déclaration des sous-programmes

Cette zone se trouve également dans la mémoire programme. Elle contiendra tous les sous-programmes que vous définirez.

La syntaxe d'un sous-programme est la suivante :

nom : ; nom du sous-programme

INST1

INST2

...

bx lr ; instruction de fin de sous-programme

Cette structure suppose que l'adresse de retour a été sauvegardée dans le registre LR.

```

7 *****/
3 *
3 *          AREA TO DECLARE SUBPROGRAMS
3 *
3 *****/
1 dosomething:
2     movs r0,#0xAB
3     bx lr
5
5 *****/
7 *
3 *          END    AREA TO DECLARE SUBPROGRAMS
3 *
3 *****/

```

2.2.1.6. Programme principal

Ce programme principal se trouve dans la mémoire programme. **C'est le point de départ du programme.**

Le programme démarre au reset et initialise le pointeur de pile. Ceci est généré lors de la création du projet et aucun changement n'est nécessaire.

```
.section .text.Reset_Handler
.weak Reset_Handler
.type Reset_Handler, %function
Reset_Handler:
    ldr    r0, =_estack
    mov    sp, r0          /* set stack pointer */

/* Copy the data segment initializers from flash to SRAM */
    ldr r0, =_sdata
    ldr r1, =_edata
    ldr r2, =_sidata
    movs r3, #0
    b LoopCopyDataInit
```

Après l'initialisation

Sa syntaxe est la suivante :

Main :

```
Instructions // initialisations diverses des périphériques (horloge, timer, ports E/S...)
boucl ...
...
b      boucl ; boucle sur une partie du programme
```

```
/* *****
 *
 *           Main Program
 *
 * *****
 */

main:
LoopForever:
    b LoopForever
```

L'instruction "b boucl" effectue un saut dans le programme principal. Celui-ci boucle donc à l'infini. Ceci doit toujours être le cas. En effet un μ contrôleur est en général embarqué dans un système qui fonctionne en permanence tant qu'il est sous tension. Le programme principal doit donc faire en sorte qu'il ne s'arrête jamais grâce à une boucle infinie.

En général, la boucle "infinie" s'effectue par un saut vers une position qui se trouve après la phase d'initialisation.

2.2.1.7. Zone de déclaration des sous-programmes d'interruption

Cette zone se trouve dans la mémoire programme. Elle contiendra tous les sous-programmes d'interruption que vous définirez.

Ces sous-programmes d'interruption sont les sous-programmes à exécuter lorsqu'une interruption (IT) provenant d'un périphérique ou d'un port du μ contrôleur survient.

La syntaxe d'un sous-programme d'interruption est la suivante :

```

/*****
*
*          Area to Declare interrupt sub-programs
*
*****/
testfunc:
// Instructions
    bx lr

/*****
*
*          END Area to Declare interrupt sub-programs
*
*****/

```

Le nom des programmes d'interruption est imposé par ARM à des fins de portabilité entre tous les fabricants de microcontrôleurs basés sur le coeur Cortex M. Lors de la création d'un projet, des fonctions sont déjà créées dans la section weak aliases

```

/*****
*
* Provide weak aliases for each Exception handler to the Default_Handler.
* As they are weak aliases, any function with the same name will override
* this definition.
*
*****/
{
    .weak    NMI_Handler
    .thumb_set NMI_Handler,Default_Handler
}

```

Toutes les interruption appellent un default_Handler.

Par simplification, pour appeler un programme d'interruption, changer le default_handler par le nom de la fonction qui doit être appelée par l'interruption

```

    .weak    EXTI15_10_IRQHandler
    .thumb_set EXTI15_10_IRQHandler,testfunc

```

2.2.1.8. Zone de déclaration des vecteurs d'interruption

Cette zone se trouve dans la mémoire programme. Les vecteurs d'interruption sont en fait les liens entre l'interruption elle-même et le sous-programme d'interruption correspondant.

```

/*****
*
* The STM32L152RETx vector table. Note that the proper constructs
* must be placed on this to ensure that it ends up at physical address
* 0x0000.0000.
*
*****/
.section .isr_vector,"a",%progbits
.type g_pfnVectors, %object
.size g_pfnVectors, .-g_pfnVectors

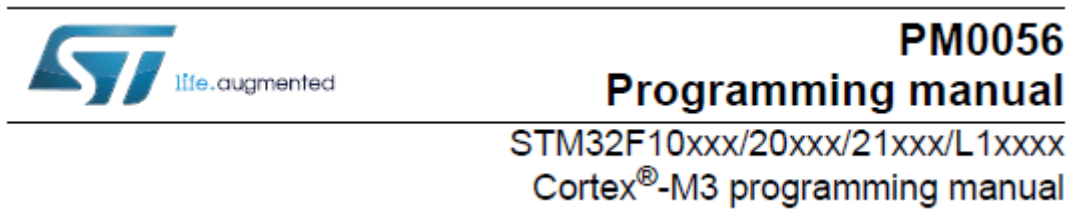
g_pfnVectors:
.word _estack
.word Reset_Handler
.word NMI_Handler
.word HardFault_Handler

```

Ces noms sont imposés par ARM. Il ne faut rien changer à cette zone.

2.3. Instructions du STM32

L'ensemble des instructions est présenté dans le Programming Manual du STM32 L152.



L'explication de l'encodage des instructions est expliqué dans le ARMv7-M Architecture Reference Manual.

ARM[®]v7-M Architecture Reference Manual

2.4. Modes d'adressage du STM32

La famille STM32 possède divers modes d'adressage. Toutes les instructions n'acceptent pas forcément tous les modes d'adressage, il faut donc vérifier dans l'aide quel mode convient.

Lors de vos TP, vous aurez souvent à manipuler des valeurs en décimal, en binaire mais aussi en hexadécimal. En assembleur, cela se traduit ainsi :

- pour coder en décimal la valeur 10, on écrira : **10**
- pour coder en binaire la valeur 10, on écrira : **0b00001010**
- pour coder en hexadécimal la valeur 10, on écrira : **0x0A**

2.4.1. Mode immédiat

Pour charger un registre avec une valeur fixe, cette valeur peut être incorporée à l'instruction. Dans ce cas, la valeur est stockée dans le programme et il n'y a pas besoin de donner l'adresse de la donnée à lire, l'adresse à pointer sera calculée directement par le compilateur.

Par exemple :

```
.equ Ndoigts,10
.equ Mask,    0b10100011
...
LDR  r0,=Ndoigts // r0 est chargé avec la valeur 10
MOV  r1,#0x10    // r1 est chargé avec la valeur décimale 0x10
MOV  r2,#Mask     ; r2 est chargé avec la valeur binaire 1010 0011 soit A3h
```

2.4.2. Mode Register

Dans ce mode, l'adresse où est stockée la valeur qui nous intéresse est enregistrée dans le registre rn. La valeur est lue en mémoire et stockée dans le registre rd.

Syntaxe : LDR rd,[rn]

Par exemple :

LDR r1,[r0] ; r0 est chargé avec la valeur du mot situé à l'adresse enregistrée dans r0

2.4.3. Mode Register avec offset immédiat

Dans ce mode, l'adresse où est stockée la valeur qui nous intéresse est enregistrée dans le registre rn incrémenté de l'offset. La valeur est lue en mémoire et stockée dans le registre rd.

Syntaxe : LDR rd,[rn,#offset]

Exemple pour lecture dans un tableau

```
varindata2:
    .word 0xFFFFFFFF, 0xDEADBEEF, 0xA

movs r0, #0b1010
ldr r0, =varindata2
ldr r1, [r0,#4]
```

A la fin de code, la valeur stockée dans r1 vaut 0xDEADBEEF

2.4.4. Mode Register avec registre d'offset

Ce mode est similaire au mode précédent.

Syntaxe : LDR rd,[rn,rm]

```
varindata2:
    .word 0xFFFFFFFF, 0xDEADBEEF, 0xA

main:
movs r0, #0b1010
ldr r0, =varindata2
ldr r2, =0x4
ldr r1, [r0,r2]
```

A la fin de code, la valeur stockée dans r1 vaut 0xDEADBEEF

2.4.12 Instructions

Afin de chercher les instructions, référez-vous au Programming Manual qui explique toutes les instructions acceptées par le STM32L152.

2.5. Ensemble de développement

La carte de développement est la NUCLEO-L152RE. Toute la documentation utile est disponible sur le site web de STMicroelectronics

https://www.st.com/content/st_com/en/products/evaluation-tools/product-evaluation-tools/mcu-mpu-eval-tools/stm32-mcu-mpu-eval-tools/stm32-nucleo-boards/nucleo-l152re.html

Le logiciel de développement utilisé est le STM32CubeIDE qui a l'avantage de fonctionner sur Windows, MAC et Linux. Il est gratuit et sans limite de taille mémoire.

2.5.1. Création d'un projet

Un document d'aide à la création d'un projet STM32 dans l'environnement de développement (IDE) STM32CubeIDE a été rédigé. Référez-vous à ce document pour chaque nouveau programme. Vous le trouverez en annexe.

L'écriture du code en assembleur et en C se fait dans l'IDE avec un éditeur à coloriage syntaxique. La compilation et l'édition de lien se font dans le même environnement ainsi que le débogage. Notez que Le STM32CubeIDE ne possède pas de simulateur et la carte est donc nécessaire pour tester un programme.

2.5.2. Utilisation du débogueur

Le STM32CubeIDE possède de nombreuses fonctionnalités qui ne peuvent pas être toutes enseignées. Néanmoins, l'annexe 3 et le chapitre 3 vous montre les fenêtres indispensables à utiliser pour bien développer un programme. Lors de la compilation, lisez les messages d'erreur. Lors du débogage, utilisez tous ces outils à votre disposition pour bien comprendre l'exécution de votre code.

Les enseignants présents avec vous sont également là pour vous aider à prendre en main cet environnement, donc n'hésitez pas à poser des questions.

Chapitre 3. Premiers pas

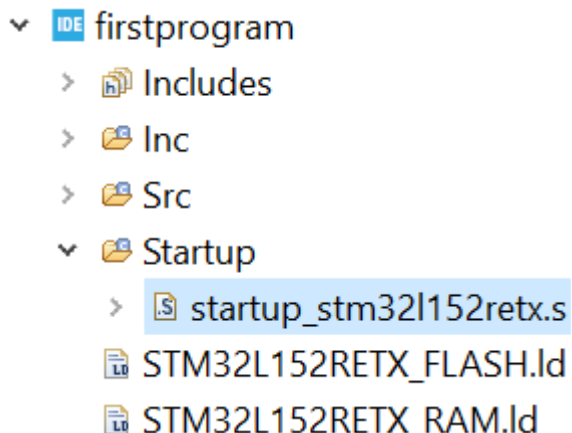
3.1. Introduction

Les premières étapes consistent à organiser l'espace de travail puis à mettre en place les éléments vous permettant de créer votre premier programme "assembleur".

En premier lieu, suivre les étapes décrites dans l'annexe 2 "Créer un projet STM32 en assembleur".

Après avoir créé un projet nous allons vous montrer comment écrire un premier programme et comment utiliser les fenêtres de débogage.

Dans votre projet, double-cliquer pour ouvrir le fichier startup_stm32l152retx.s



Repérer, comme expliqué pendant le cours, les différentes zones pour écrire le programme. Par exemple, la zone pour déclarer des variables initialisées

```
13
76 /*****
77 *
78 *          AREA TO DECLARE INITIALIZED VARIABLES
79 *
80 /*****
81 .section .data
82
83
84
85 /*****
86 *
87 *          END    AREA TO DECLARE INITIALIZED VARIABLES
88 *
89 /*****
--
```

Pour déclarer une variable sur 32 bits (par souci de simplification, nous ne déclarerons que des variables 32-bit pour les TP en assembleur). La syntaxe est la suivante :

Nom de variable :

.word valeur de la variable

Donc pour déclarer la variable de nom 'maVariable' initialisée à 2, le code est écrit comme ci-dessous

```

76 /*****
77 *
78 *          AREA TO DECLARE INITIALIZED VARIABLES
79 *
80 /*****
81 .section .data
82
83 maVariable:
84     .word    2
85
86
87 /*****
88 *
89 *          END    AREA TO DECLARE INITIALIZED VARIABLES
90 *
91 /*****/

```

Nous allons faire un programme qui récupère l'adresse de la variable maVariable au début du programme puis soustrait 1 dans la boucle infinie.

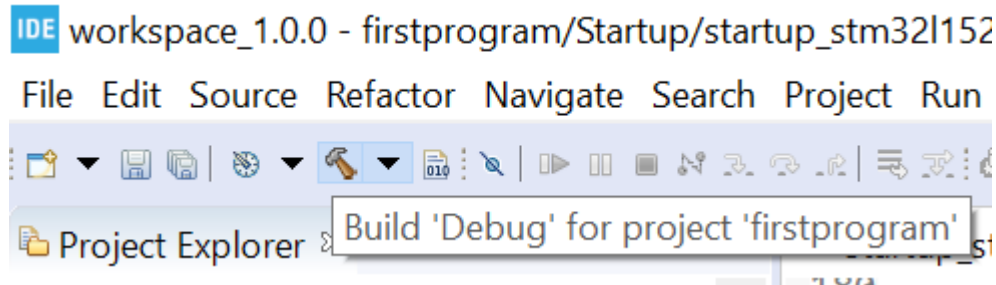
Ecrivez le programme suivant :

```

7 main:
3
3 ldr r0, =maVariable // récupère l'adresse de la variable r0
3 ldr r1, [r0] // mets la valeur pointée par r0 dans r1
1
2 LoopForever:
3 // soustrait 1 à r1 en mettant à jour le registre xPSR
4     subs r1, #1
5
5     b LoopForever
7
3 .size Reset_Handler, .-Reset_Handler
3

```

Compilez le programme

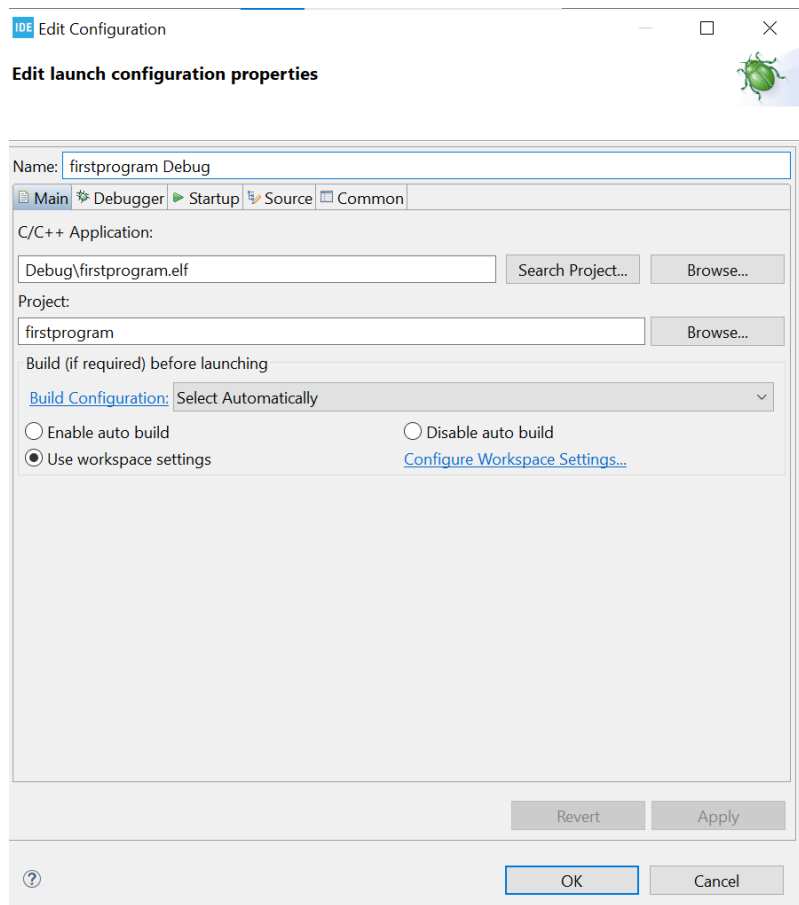


La compilation donne des informations sur la taille mémoire du programme, la taille de SRAM utilisée et les erreurs éventuelles qui empêchent la compilation

```
arm-none-eabi-objdump -h -S firstprogram.elf > "firstprogram.list"
arm-none-eabi-objcopy -O binary firstprogram.elf "firstprogram.bin"
arm-none-eabi-size firstprogram.elf
  text    data     bss     dec     hex filename
   548      12    1564    2124    84c firstprogram.elf
Finished building: default.size.stdout
Finished building: firstprogram.list
Finished building: firstprogram.bin
```

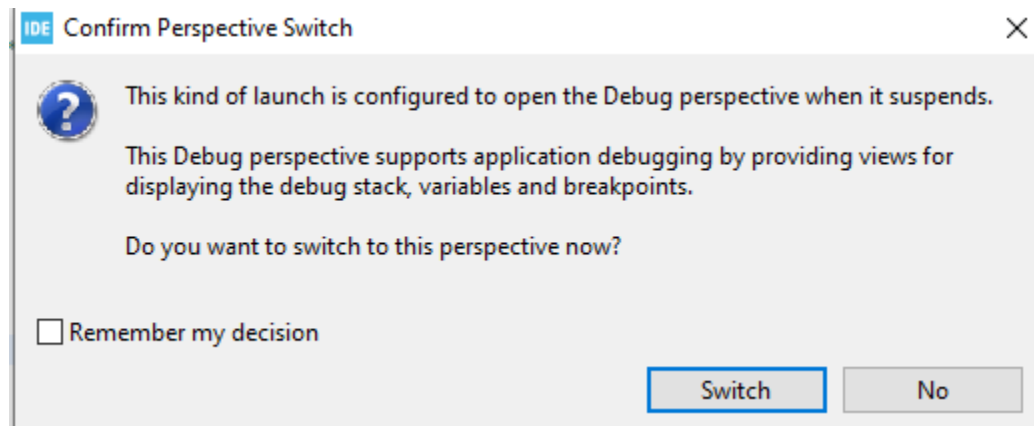
10:28:37 Build Finished. 0 errors, 0 warnings. (took 1s.81ms)

Lancer le programme en mode debug. La première fois, il est demandé de confirmer la configuration du debug.



Cliquez sur ok

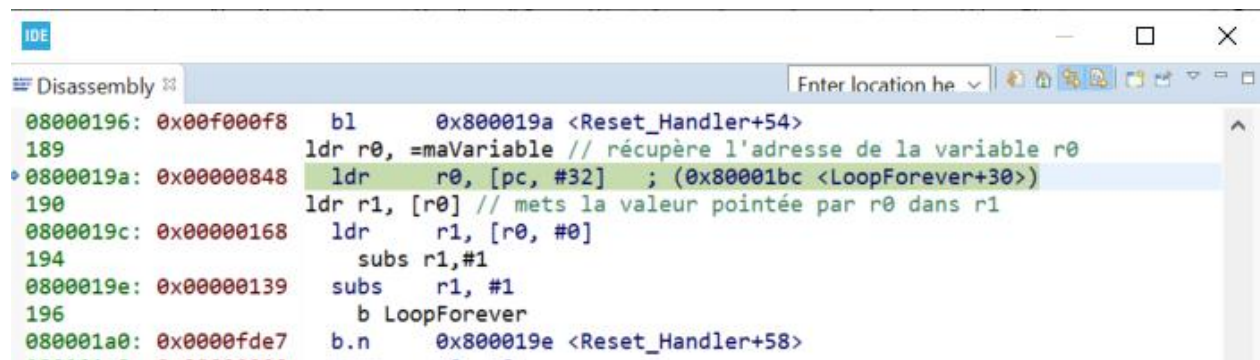
Quand l'IDE demande de changer à la perspective debug, cliquez sur switch.



Le programme s'arrête à la première ligne de code du main.

```
1
2
3 main:
4
5 ldr r0, =maVariable // récupère l'adresse de la variable r0
6 ldr r1, [r0] // mets la valeur pointée par r0 dans r1
7
8 LoopForever:
9 // soustrait 1 à r1 en mettant à jour le registre xPSR
10 subs r1,#1
11
12 b LoopForever
13
14 .size Reset_Handler, .-Reset_Handler
15
```

Ouvrir la fenêtre de désassemblage, qui permet de voir le codé écrit, le code compilé, l'encodage et l'adresse des instructions.



Grâce à la fenêtre de désassemblage, on voit que le programme est stoppé à l'adresse 0x0800019A. Bien noté que l'instruction à l'adresse 0x0800019A n'est pas encore exécutée. Le Program Counter pointe sur la prochaine instruction à exécuter au prochain coup d'horloge et non sur l'instruction qui vient d'être exécutée !

Ouvrir la fenêtre des registres.

Registers	
Name	Value
General Registers	
r0	0x20000000 (Hex)
r1	0x20000004 (Hex)
r2	0x20000020 (Hex)
r3	0x8000149 (Hex)
r4	0x20000020 (Hex)
r5	0x0 (Hex)
r6	0x0 (Hex)
r7	0x0 (Hex)
r8	0x0 (Hex)
r9	0
r10	0
r11	0
r12	0
sp	0x20014000
lr	0x800019b (Hex)
pc	0x800019a <Reset_Handle...
xpsr	0x61000000 (Hex)

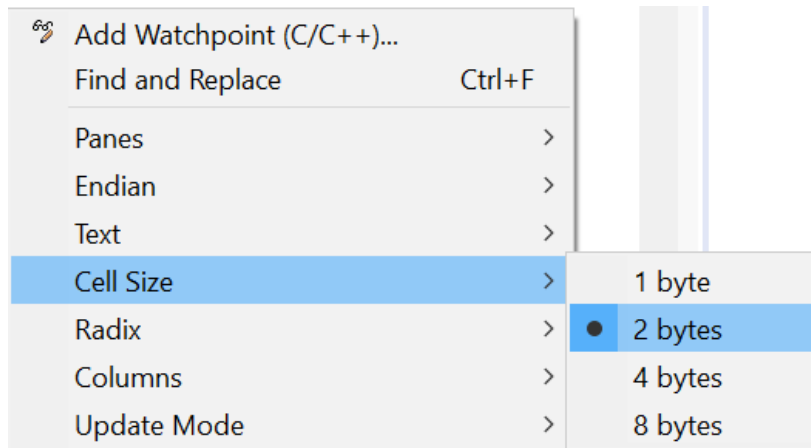
La valeur du Program Counter est bien 0x0800019A. La prochaine instruction à exécuter sera :
ldr r0, =mavariab

Ouvrir la fenêtre Memory Browser et faire apparaître la mémoire à l'adresse 0x0800019A

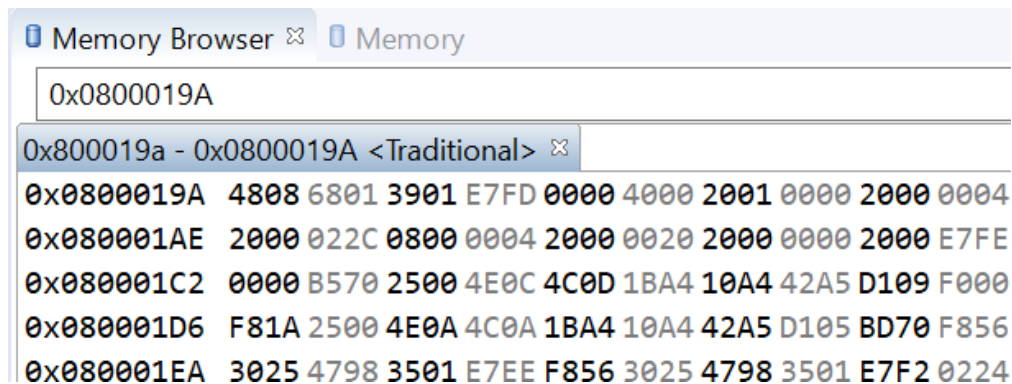
Memory Browser	
Memory	
0x0800019A	
0x800019a - 0x0800019A <Traditional>	
0x0800019A	68014808 E7FD3901 40000000 00002001 00042000
0x080001AE	022C2000 00040800 00202000 00002000 E7FE2000
0x080001C2	B5700000 4E0C2500 1BA44C0D 42A510A4 F000D109
0x080001D6	2500F81A 4C0A4E0A 10A41BA4 D10542A5 F856BD70
0x080001EA	47983025 E7EE3501 3025F856 35014798 0224E7F2

Par défaut, la présentation est faite par blocs de 32 bits. Comme les instructions sont codées sur 16 bit, il peut être plus lisible, pour étudier l'encodage, de faire un affichage sur 16 bits.

Placer le curseur sur la fenêtre Memory browser et cliquer sur le bouton droit de la souris.
Sélectionner la taille de la cellule sur 2 octets.



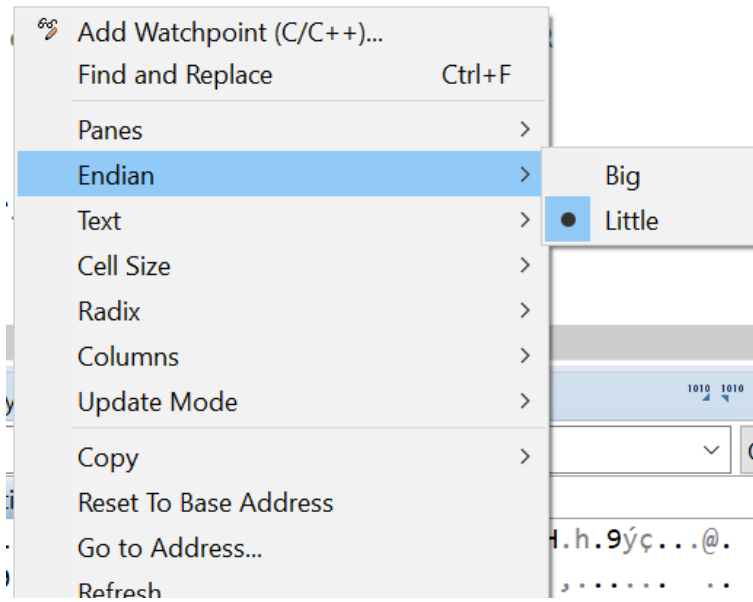
La fenêtre est alors organisée comme suit



On voit qu'à l'adresse 0x0800019A, en mémoire, nous avons l'encodage 0x4808 alors que la fenêtre de désassemblage donne 0x0848.

Le STM32L152 est configuré en little endian.

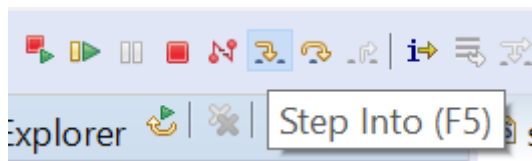
Mettre le curseur sur la fenêtre Memory Browser, cliquer sur le bouton droit et sélectionner little endian. L'affichage est bien en little endian



Si on passe à une présentation par octet, on voit bien qu'on a 0x08 puis 0x48 en mémoire

```
0x800019a - 0x0800019A <Traditional> x
0x0800019A 08 48 01 68 01 39 FD E7 00 00 00 40 01 20 00 00 00 20 04 00
0x080001AE 00 20 2C 02 00 08 04 00 00 20 20 00 00 20 00 00 00 20 FE E7
0x080001C2 00 00 70 B5 00 25 0C 4E 0D 4C A4 1B A4 10 A5 42 09 D1 00 F0
0x080001D6 1A F8 00 25 0A 4E 0A 4C A4 1B A4 10 A5 42 05 D1 70 BD 56 F8
0x080001EA 25 30 98 47 01 35 EE E7 56 F8 25 30 98 47 01 35 F2 E7 24 02
0x080001FE 00 08 24 02 00 08 24 02 00 08 28 02 00 08 F8 B5 00 BF F8 BC
```

Pour exécuter une instruction, faire un step into en cliquant sur l'icône flèche



L'adresse de maVariable est copiée dans r0. La valeur de r0 vaut 0x20000000.

Grâce au memory browser aller à l'adresse 0x20000000. La valeur 2 est bien stockée à cet emplacement mémoire

```
0x20000000 <Traditional> x
0x20000000 00000002 00000000 00000000 00000000 00000000 00000000
0x20000018 00000000 00000000 02C06800 F245D40D F8DF5055 600815E8
0x20000030 F8DF2006 600815E4 70FFF640 15DCF8DF 20016008 B5F8BD02
```


Pour récupérer la valeur 2, exécuter l'instruction grâce au step into
 La valeur du registre r1 vaut alors 2 ; remarque que le PC change également.

Name	Value
▼  General Registers	
 r0	0x20000000 (Hex)
 r1	0x2 (Hex)
 r2	0x20000020 (Hex)
 r3	0x8000149 (Hex)
 r4	0x20000020 (Hex)
 r5	0x0 (Hex)
 r6	0x0 (Hex)
 r7	0x0 (Hex)
 r8	0x0 (Hex)
 r9	0
 r10	0
 r11	0
 r12	0
 sp	0x20014000
 lr	0x800019b (Hex)
 pc	0x800019e <Reset_Handle.
 xpsr	0x61000000 (Hex)
 PRIMASK	0
 BASEPRI	0

Exécuter le programme pas à pas. Lorsque r0 vaut 0, xpsr vaut 0x6100 0020
 0x6 = 0b0110 et correspond aux drapeaux N, Z, C et V

Name	Value
General Registers	
r0	0x20000000 (Hex)
r1	0x0 (Hex)
r2	0x20000020 (Hex)
r3	0x8000149 (Hex)
r4	0x20000020 (Hex)
r5	0x0 (Hex)
r6	0x0 (Hex)
r7	0x0 (Hex)
r8	0x0 (Hex)
r9	0
r10	0
r11	0
r12	0
sp	0x20014000
lr	0x800019b (Hex)
pc	0x80001a0 <Reset_Handle.
xpsr	0x61000020 (Hex)

Continuer le programme pour obtenir 0xFFFF FFFF dans r1. Vérifier comment xPSR a été modifié. Reprendre le cours pour en comprendre la signification.

Name	Value
General Registers	
r0	0x20000000 (Hex)
r1	0xffffffff (Hex)
r2	0x20000020 (Hex)
r3	0x8000149 (Hex)
r4	0x20000020 (Hex)
r5	0x0 (Hex)
r6	0x0 (Hex)
r7	0x0 (Hex)
r8	0x0 (Hex)
r9	0
r10	0
r11	0
r12	0
sp	0x20014000
lr	0x800019b (Hex)
pc	0x80001a0 <Reset_Handle.
xpsr	0x81000020 (Hex)
PRIMASK	0
BASEPRI	0
FAULTMASK	0

Name : xpsr
Hex: 0x81000020
Decimal: -2130706400
Octal: 020100000040
Binary: 10000001000000000000000000000000
Default: -2130706400

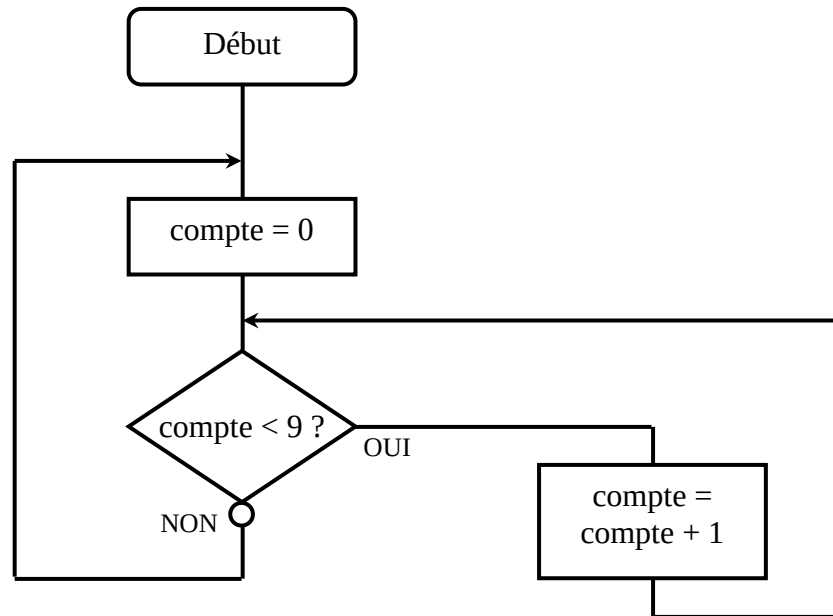
Pour bien déboguer les programmes, il faudra suivre une démarche similaire pour trouver vos erreurs. A vous de jouer maintenant.

3.2. Réalisation d'un compteur modulo 10

Cet exercice consiste à réaliser un programme qui compte de 0 à 9 puis recommence à 0. Ce programme boucle indéfiniment.

Le compteur sera mémorisé dans une variable nommée "compte" que vous devrez définir dans la zone des variables. Vous utiliserez une séquence répétitive du type "tant que ... faire".

L'organigramme suivant illustre l'architecture de ce programme :



Vous devez :

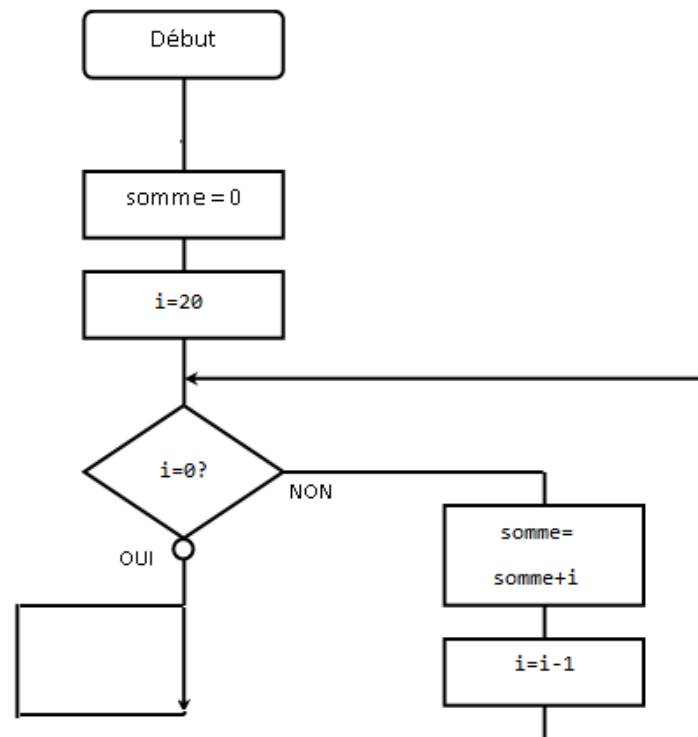
- Ecrire le programme en respectant l'organigramme ci-dessus.
- Compiler le programme.
- Tester le programme avec le débogueur en mode pas à pas afin de vérifier son bon fonctionnement en visualisant l'évolution des registres R0, R1 et l'état de la variable compte.

3.3. Comment faire une boucle "for" ?

Cet exercice consiste à réaliser, en assembleur, une boucle "for", connue dans les langages évolués. Pour cela, on vous demande de calculer la somme des 20 premiers entiers (1 à 20).

La somme sera mémorisée dans une variable d'un octet nommée "somme" que vous devrez définir dans la zone des variables. Par contre, l'indice de la boucle "i" ne sera pas déclaré comme une variable, mais utilisera un registre.

L'organigramme suivant illustre l'architecture de ce programme :



Vous devez :

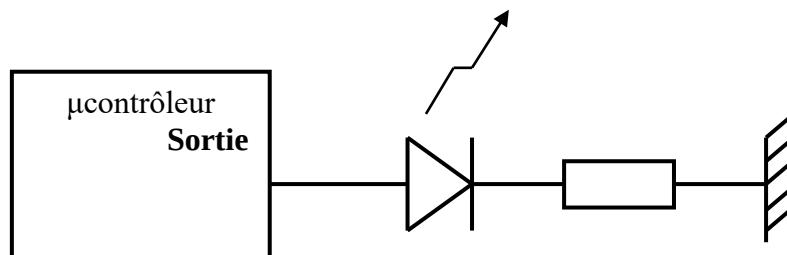
- Calculer la somme finale attendue pour juger le fonctionnement du programme.
- Ecrire le programme en assembleur en respectant l'organigramme ci-dessus.
- Tester le programme avec le débogueur en mode pas à pas afin de vérifier son bon fonctionnement en:
 - Visualisant la fenêtre "désassembleur".
 - Visualisant la fenêtre "mémoire".
 - Situait dans la mémoire, la variable "somme" et le programme.
 - l'évolution des registres R0, R1 et l'état de la variable compte.

3.4. Mixer assembleur et C, Clignotement d'une LED avec ODR et BSRR

Il est bien évidemment possible d'écrire dans un programme C des instructions assembleur. C'est parfois nécessaire si une partie du programme nécessite par exemple la plus grande rapidité possible. C'est aussi utilisé pour faire des temporisations sans utiliser les fonctions HAL ou les interruptions timer.

Cet exercice consiste à faire clignoter la LED1 connectée à PB0. Les fonctions allume() et éteint() seront écrites en assembleur dans un fichier source asm.

La LED est commandée par le bit 0 du port B en sortie *push-pull* comme l'indique la figure suivante :



Vous créerez un projet STM32 en langage C. La broche PB0 sera configurée avec STM32CubeMX. Puis:

- Dans le while(1) de la fonction main(), vous ajouterez les instructions de clignotement

```
allume();  
HAL_Delay(499);  
eteint();  
HAL_Delay(499);
```

- Dans le dossier Core/Inc, ajouter le fichier fonc_asm.h et ouvrez-le pour voir la déclaration des fonctions
- En début du fichier main.c, ajouter #include fonc_asm.h
- Dans le dossier Core/ Src, ajouter le fichier fonc_asm.s
- Dans le fichier fonc_asm.s, sous les étiquettes allume et éteint, ajouter les instructions assembleur pour modifier l'état de la LED en utilisant l'ODR
- Testez votre programme afin de vérifier son bon fonctionnement.
- Dans le fichier fonc_asm.s, sous les étiquettes allume et éteint, modifier les instructions assembleur pour utiliser le BSRR
- Testez votre programme afin de vérifier son bon fonctionnement.
- Que déduisez-vous des 2 méthodes utilisées ?

Annexe 1. Les instructions saut conditionnel

cond	Mnemonic extension	Meaning (integer)	Meaning (floating-point) ^{ab}	Condition flags
0000	EQ	Equal	Equal	$Z == 1$
0001	NE	Not equal	Not equal, or unordered	$Z == 0$
0010	CS ^c	Carry set	Greater than, equal, or unordered	$C == 1$
0011	CC ^d	Carry clear	Less than	$C == 0$
0100	MI	Minus, negative	Less than	$N == 1$
0101	PL	Plus, positive or zero	Greater than, equal, or unordered	$N == 0$
0110	VS	Overflow	Unordered	$V == 1$
0111	VC	No overflow	Not unordered	$V == 0$
1000	HI	Unsigned higher	Greater than, or unordered	$C == 1$ and $Z == 0$
1001	LS	Unsigned lower or same	Less than or equal	$C == 0$ or $Z == 1$
1010	GE	Signed greater than or equal	Greater than or equal	$N == V$
1011	LT	Signed less than	Less than, or unordered	$N != V$
1100	GT	Signed greater than	Greater than	$Z == 0$ and $N == V$
1101	LE	Signed less than or equal	Less than, equal, or unordered	$Z == 1$ or $N != V$
1110	None (AL) ^e	Always (unconditional)	Always (unconditional)	Any

Annexe 2. Créer un projet STM32

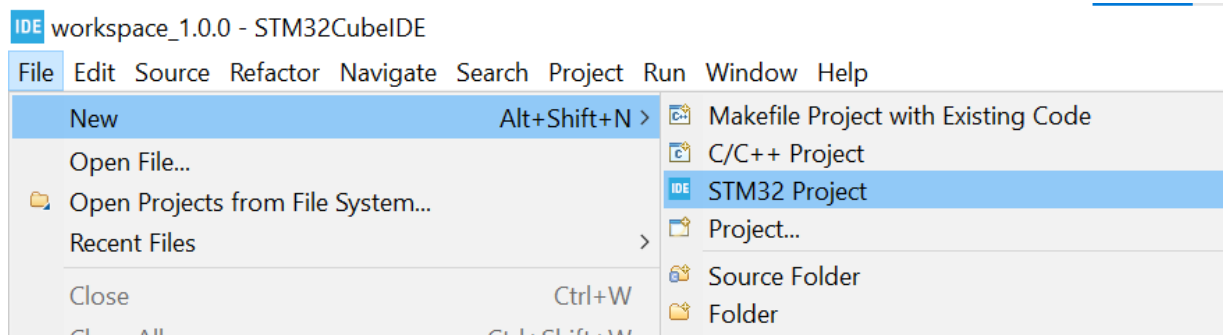
Le kit de développement de STMicroelectronics dédié aux microcontrôleurs des familles STM32 permet de créer très simplement des projets avec un environnement graphique.

Ce livret vous guidera pas à pas dans la création d'un tel projet.

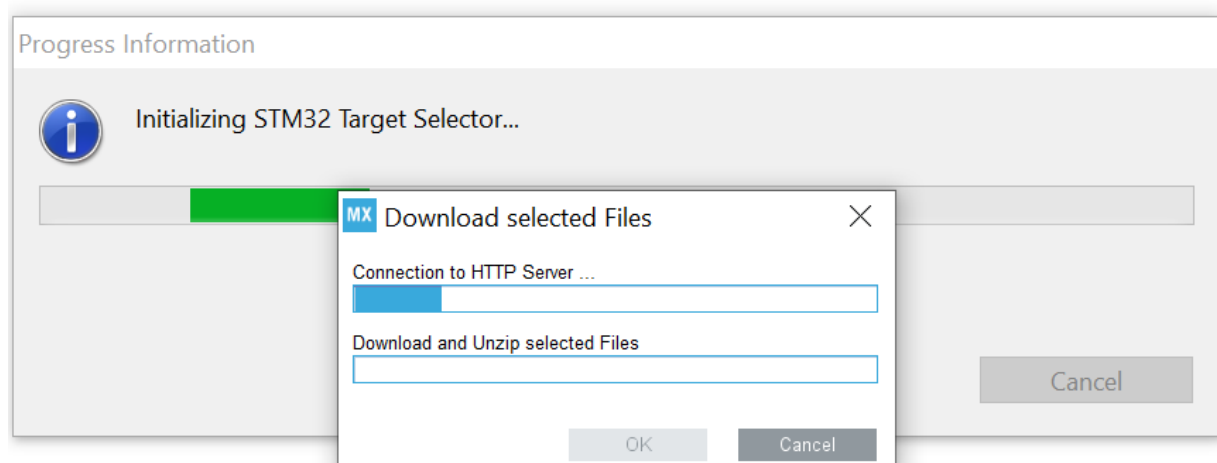
Chaque étape sera détaillée et illustrée de captures d'écran afin de faciliter votre lecture.

Après avoir lu ce petit guide, vous serez capable de créer votre premier projet et de le tester

Pour créer un nouveau projet, cliquer sur File/New/STM32 Project



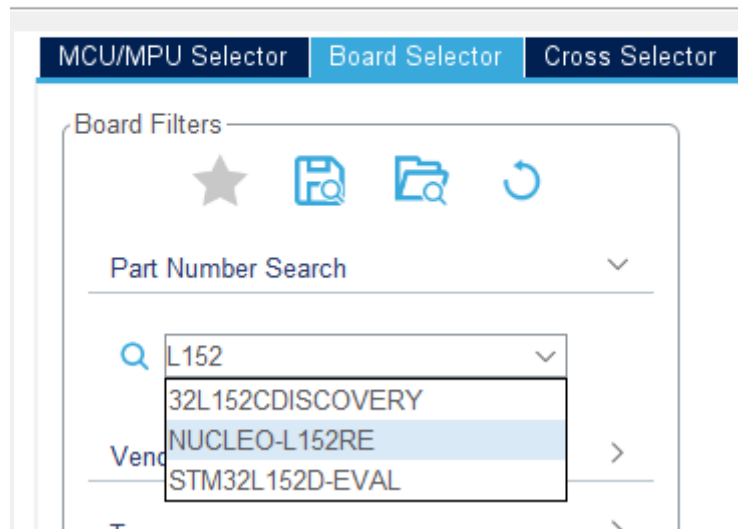
Si une connexion à internet est disponible, STM32CubeIDE peut mettre à jour l'offre des produits disponibles. Cette mise à jour peut prendre quelques secondes.



Sélectionner la tabulation Board Selector et dans la fenêtre de recherche Part Number Research, sélectionner Nucleo-L152RE

Target Selection

Select STM32 target



A droite de la fenêtre, la carte utilisée pour le TP apparaît.

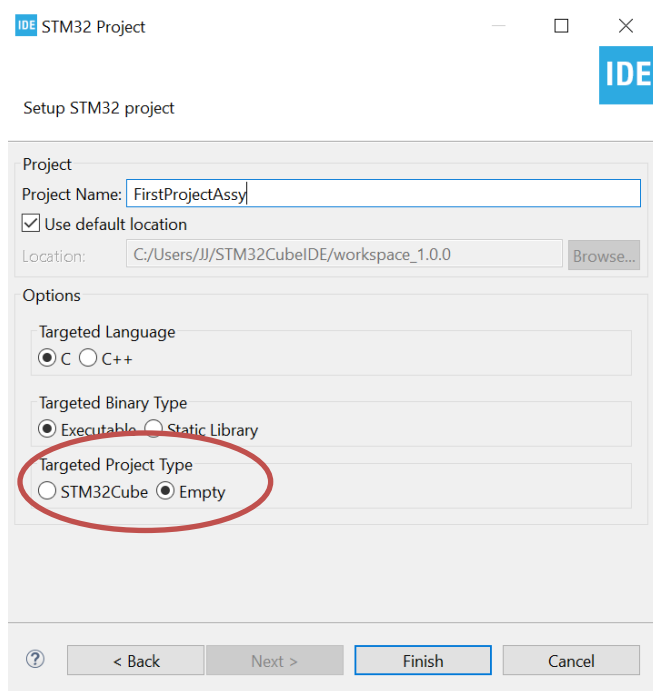
Boards List: 1 item Export

	Overview	Part No.	Type	Marketing Status	Unit Price (US\$)	Mounted Device
		NUCLEO-L152RE	Nucleo64	Active	13.0	STM32L152RETx

Entrer le nom du projet.

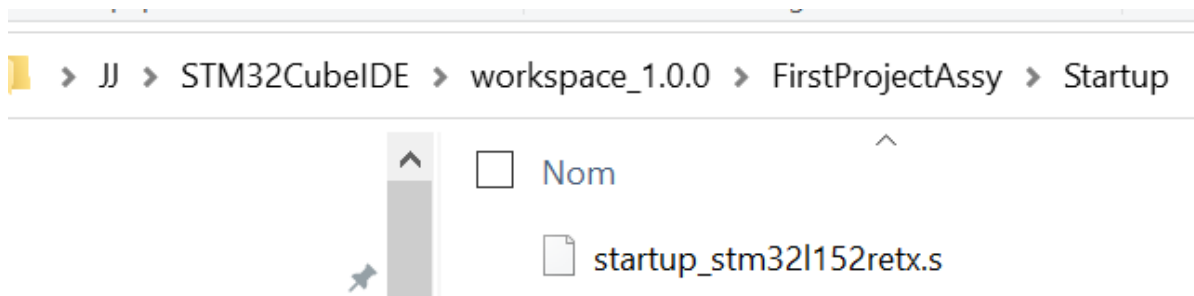
!!! Dans Targeted Project Type, sélectionner Empty !!!

Dans le cas contraire, des bibliothèques inutiles aux TP en assembleur seront ajoutées au projet



Appuyez sur Finish.

Avec Windows Explorer dans le workspace que vous avez créé et dans le dossier start up.

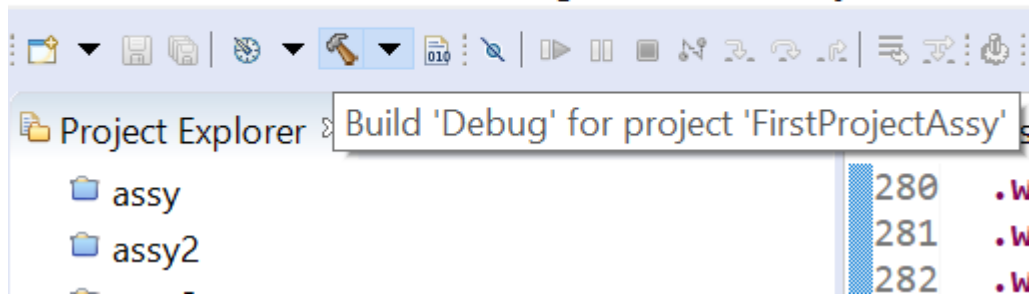


Attention : Remplacer le fichier startup_stm32l152retx.s par le fichier du même nom publié sur l'ENT.

Puis compiler votre programme

IDE workspace_1.0.0 - FirstProjectAssy/Startup/startup_stm32l15

File Edit Source Refactor Navigate Search Project Run V



```
arm-none-eabi-size FirstProjectAssy.elf
arm-none-eabi-objdump -h -S FirstProjectAssy.elf > "FirstProjectAssy.list"
text    data    bss     dec      hex filename
  536      8   1568   2112    840 FirstProjectAssy.elf
arm-none-eabi-objcopy -O binary FirstProjectAssy.elf "FirstProjectAssy.bin"
Finished building: default.size.stdout
```

Finished building: FirstProjectAssy.list

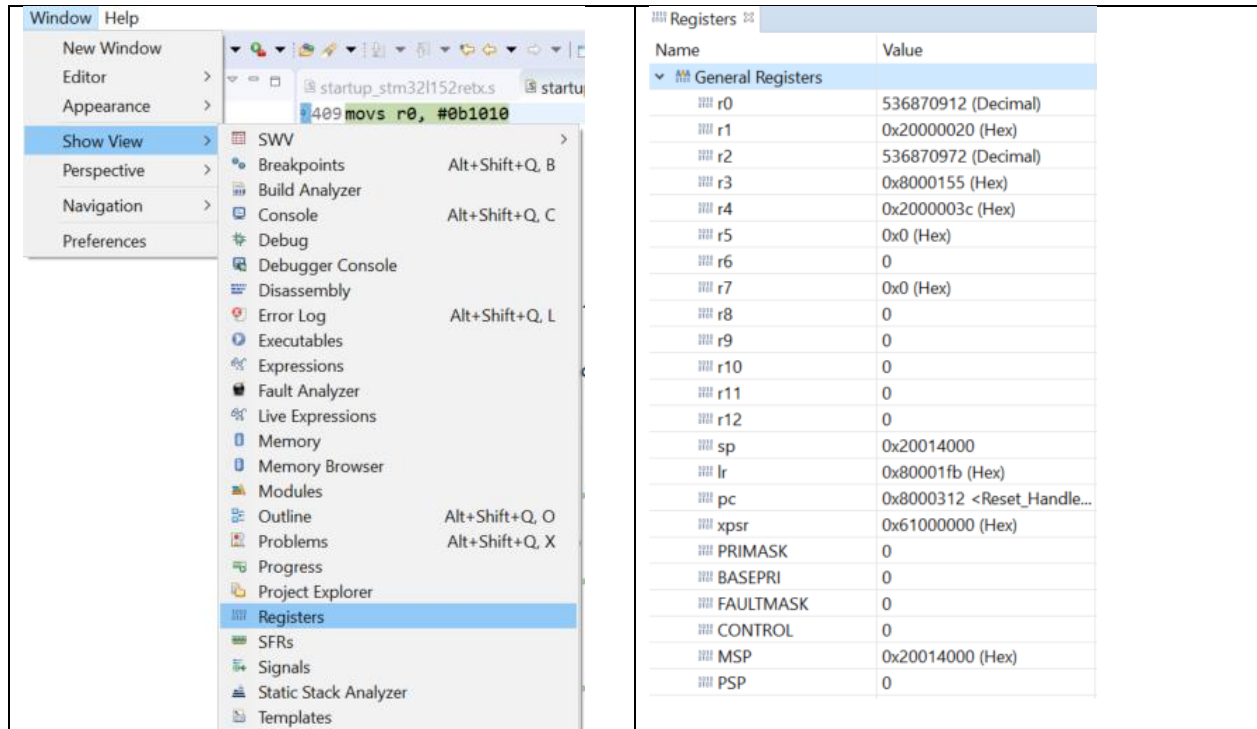
Finished building: FirstProjectAssy.bin

10:29:05 Build Finished. 0 errors, 0 warnings. (took 1s.491ms)

Vous êtes maintenant prêts à écrire vos programmes en assembleur. Les fenêtres présentées en annexe 3 vous seront nécessaires pour répondre aux questions du TP et déboguer votre programme.

Annexe 3. Fenêtre de débogage

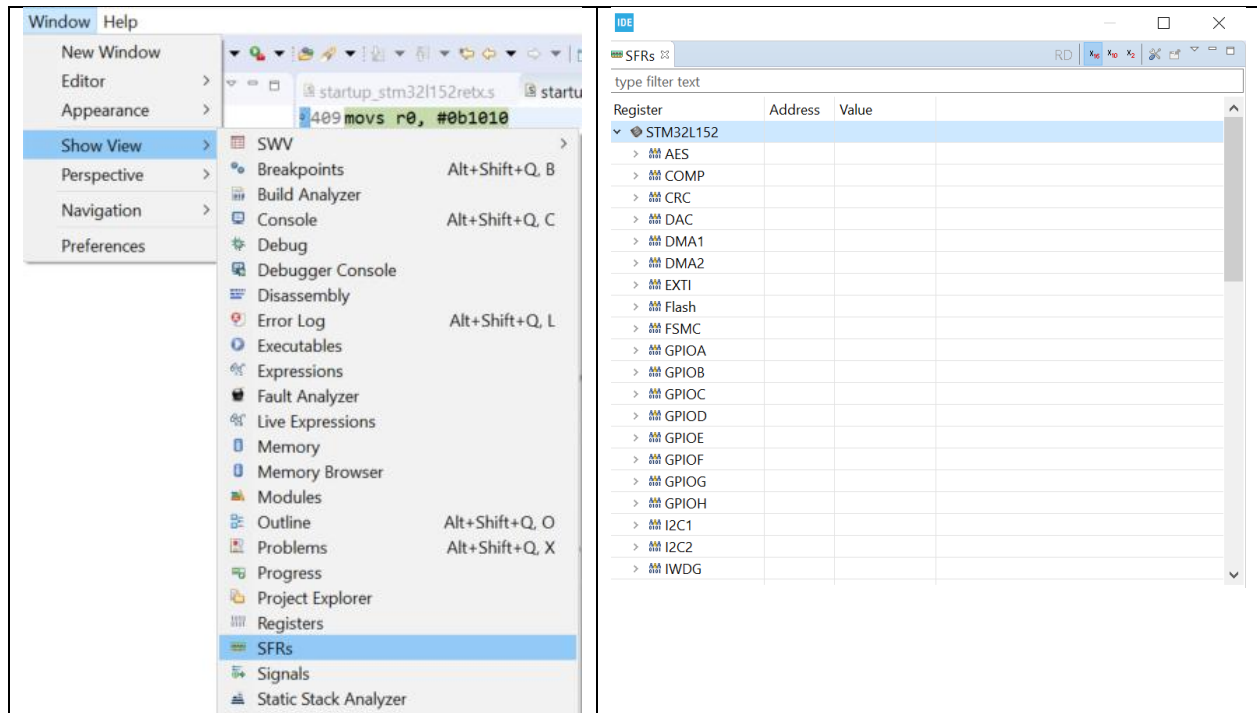
Fenêtre pour accéder aux registres internes



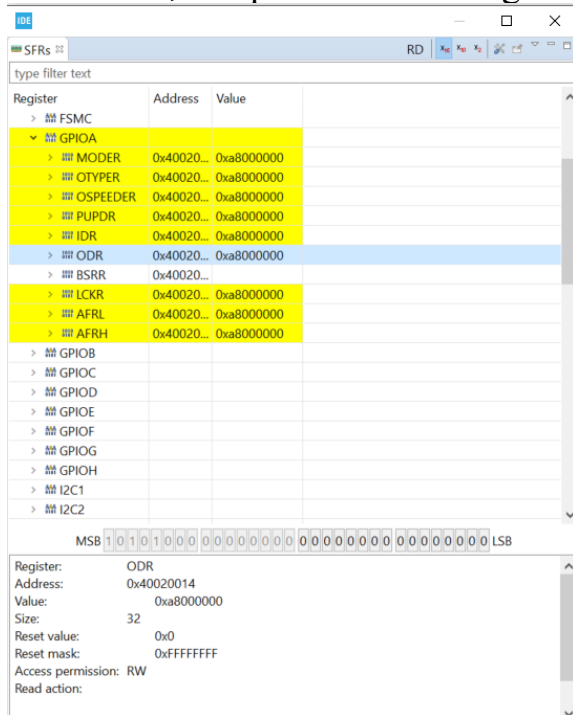
The screenshot shows the 'Window' menu on the left, with 'Registers' selected. The 'Registers' window on the right displays a table of internal registers.

Name	Value
General Registers	
r0	536870912 (Decimal)
r1	0x20000020 (Hex)
r2	536870972 (Decimal)
r3	0x8000155 (Hex)
r4	0x2000003c (Hex)
r5	0x0 (Hex)
r6	0
r7	0x0 (Hex)
r8	0
r9	0
r10	0
r11	0
r12	0
sp	0x20014000
lr	0x80001fb (Hex)
pc	0x8000312 <Reset_Handle...
xpsr	0x61000000 (Hex)
PRIMASK	0
BASEPRI	0
FAULTMASK	0
CONTROL	0
MSP	0x20014000 (Hex)
PSP	0

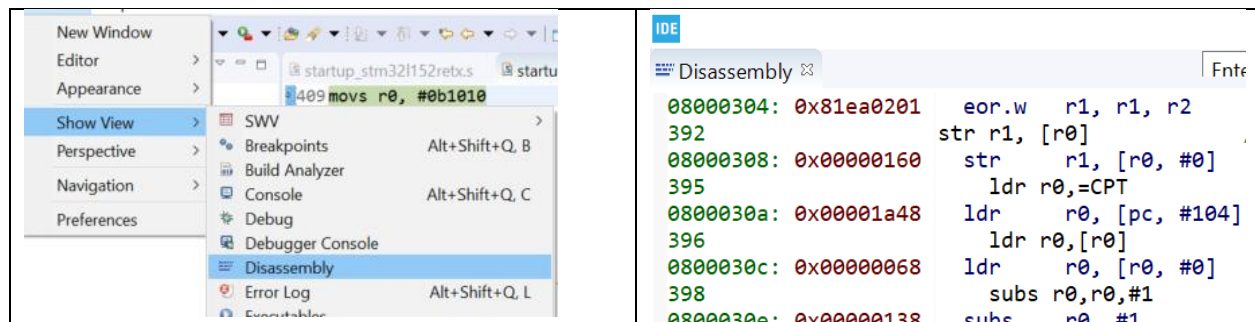
Fenêtre pour accéder aux périphériques



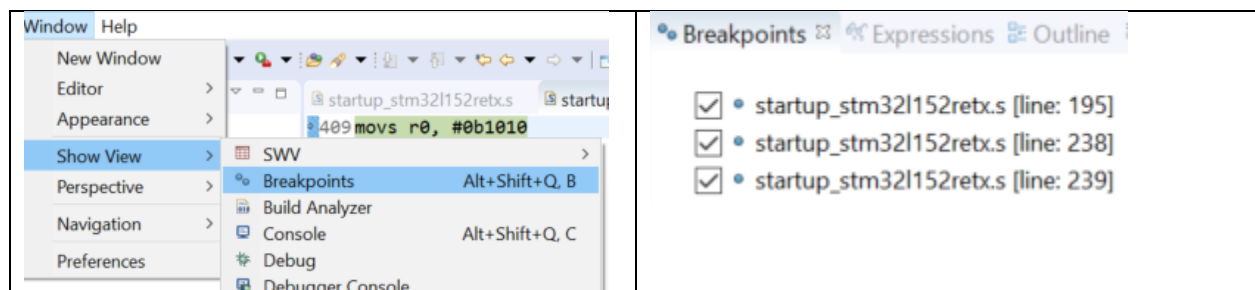
Et bien sûr, chaque détail d'un registre est accessible



Fenêtre de désassemblage



Fenêtre pour accéder aux points d'arrêt



Pour créer un point d'arrêt, double cliquer en face de la ligne. Pour annuler un point d'arrêt, dans la fenêtre breakpoint, cliquer sur un point d'arrêt et appuyer sur la touche 'suppr'.

Fenêtre pour accéder à une adresse mémoire

